

Lecture 7: The Krusell–Smith Method

Computational Methods for Heterogeneous-Agent Macro

Jeffrey Sun

May 19, 2026

University of Toronto

Curse of Dimensionality

Setup

Aggregate state. TFP $Z_t \in \{Z_{\text{bad}}, Z_{\text{good}}\}$ Markov with P_Z .

Idiosyncratic state. (z_{it}, b_{it}) Productivity $z_{it} \in \{z_{\text{lo}}, z_{\text{hi}}\}$ Markov with P_z ; Wealth b_{it} .

Technology. Cobb–Douglas with TFP Z_t :

$$Y_t = Z_t K_t^\alpha L^{1-\alpha}.$$

L is assumed-constant stationary mean of z .

Prices

Rental rate and wage given by Z_t, K_t, L ,

$$r(Z_t, K_t) = \alpha Z_t (K_t/L)^{\alpha-1} - \delta,$$

$$w(Z_t, K_t) = (1 - \alpha) Z_t (K_t/L)^\alpha.$$

Problem: Household price forecasts both **affect** and **depend on** future K_{t+1} .

K_{t+1} depends on Λ_t

$$K_{t+1} = \int b'_i(b_i, z_i; Z_t, K_t) d\Lambda_t(b_i, z_i).$$

Policy $b'_i(b_i, z_i; Z_t, K_t)$ depends on price forecasts.

Price r_{t+s}, w_{t+s} forecasts depend on Λ_{t+s} through K_{t+s} .

Note: Same K_t from different Λ_t can imply different K_{t+1} .

Bellman Equation

Distribution Λ becomes a state variable:

$$V(b_i, z_i; \Lambda, Z) = \max_{c_i, b'_i} u(c_i) + \beta \mathbb{E}[V(b'_i, z'_i; \Lambda', Z') \mid z_i, Z],$$

Λ is infinite-dimensional

Λ_t is a **distribution** over every (b, z) .

The “true” Λ_t is infinite-dimensional.

Even if we discretize Λ_t into a matrix, **every gridpoint is a state variable**.

With $N_w = 80$ wealth gridpoints and $N_z = 2$ productivity levels, grid has 160 points.

A V grid over possible Λ values: 5^{160-1} cells. About 10^{111} .

Tractably Parametrizing $V(\dots ; \dots, \Lambda)$

- (a) **Reduce Λ .** (Krusell-Smith.) Use moments of Λ_t instead of the whole distribution.
- (b) **Reduce the representation.** (nVFI.) Parameterise $V(\Lambda, \cdot)$ by a flexible family of functions (e.g. a **neural network**).

Krusell–Smith Method

Microfoundation: Bounded Rationality

Krusell and Smith (1998): Assume the **household** only observes moments of Λ_t , not the whole distribution.

i.e. solve a model of $V(b, z; Z, K)$, not $V(b, z; Z, \Lambda)$.

The Approximate Law of Motion

Assume household uses a forecast rule for K :

$$\log K_{t+1} = a_0 Z_t + a_1 Z_t \log K_t.$$

Two coefficients per Z state. For 2-state Z , four numbers total.

Log-linear. K-S find this is actually a very good fit.

The bounded-rationality Bellman

Replace Λ with K in the state:

$$V(b_i, z_i, Z, K) = \max_{c_i, b'_i} u(c_i) + \beta \mathbb{E}[V(b'_i, z'_i, Z', K')],$$

subject to

$$c_i + b'_i = (1 + r(Z, K)) b_i + w(Z, K) z_i,$$

with the agent's *belief*

$$K' = \exp(a_0(Z) + a_1(Z) \log K),$$

and Z' drawn from P_Z .

The chain in five stages

Within a period, in time order:

1. z_shock: $z_{it} \rightarrow z_i^{t+1}$ (Markov on z).
2. income: $b'_i \leftarrow (1+r)b_i + wz_i$, with (r, w) from (Z, K) .
3. savings: pick b_i^{t+1} .

Moment: K_evolve: $K_t \rightarrow K_{t+1}$. Aggregate Shock: Z_shock: $Z_t \rightarrow Z_{t+1}$.

z_shock $\circ$ income $\circ$ savings.

K-S Outer Loop

1. **Pick** an initial ALM $(a_0^{(0)}, a_1^{(0)})$.
2. **Solve** the 4D household VFI given the current ALM.
3. **Simulate** a long economy under realised $\{Z_t\}$ and *true* aggregation $K_t = \int b d\Lambda_t$.
4. **Refit** (a_0, a_1) by OLS of $\log K_{t+1}$ on $\log K_t$, separately per Z .
5. **Iterate** until ALM coefficients stop moving.

Implementing K–S

Parameters

Canonical K-S calibration (1998):

- $\beta = 0.96$, $\gamma = 1$ (log utility), $\alpha = 0.36$, $\delta = 0.025$.
- $Z \in \{0.99, 1.01\}$ with P_Z persistent (0.875 on-diagonal).
- $z \in \{0.07, 1.0\}$.

Grids (small, for pedagogical speed):

- $N_w = 80$ on log-spaced $[0, 80]$.
- $N_K = 5$ on linear $[10.5, 14.5]$.
- Cell count: $80 \times 2 \times 2 \times 5 = 1600$.

Setup: K-grid center at $Z = 1$

Before the K-S iteration starts we need:

- $\bar{K}$, the K-grid center (the agent's belief set).
- Λ_0 , the initial wealth distribution for the simulation.

Both come from the **deterministic Aiyagari SS at $Z = 1$** — the L4 tatonnement applied with Z held fixed at the midpoint of $\{Z_{\text{bad}}, Z_{\text{good}}\}$. The 2D household block (no Z , no K in the state) closes the loop on K alone.

Tatonnement converges in ~ 10 iterations to $\bar{K} \approx 13$ on this grid.

State layout

```
layout = StateLayout(  
    StateAxis(:wealth, continuous_grid(p.w_min, p.w_max;  
                                       length=p.N_w, spacing=:log)),  
    StateAxis(:z,      p.z_grid),  
    StateAxis(:Z_idx,  [1, 2]),  
    StateAxis(:K,      continuous_grid(p.K_min, p.K_max; length=p.N_K)),  
)
```

- Two continuous axes (:wealth, :K), two categorical (:z, :Z_idx).
- Same layout the simulator will use, modulo the chain we run on it.

Stages 1–2: z_shock and income

Stage 1–2

```
z_shock = MarkovStage(layout; axis=:z, transition=p.P_z)

income = WealthChangeStage(layout;
    wealth_post=(cell; env) -> begin
        Z = p.Z_vals[cell.Z_idx]
        pr = ks_prices(cell.K, Z, p)
        (1 + pr.r) * cell.wealth + pr.w * cell.z
    end,
)
```

Note. The income closure reads (Z, K) from the cell coordinates, not from env. So prices vary across cells *within* a single solve.

Stage 3: Consumption Savings

Stage 3

```
savings = ConsumptionSavingsStage(layout;β
    =pβ.,
    utility=(cell, c; env) -> u_crra(c, Val(py.)),
    monotone_search=:divide_conquer,
)
```

- Same stage as Aiyagari / spatial.
- Utility is per-cell — but here only depends on c .
- Divide-and-conquer search exploits monotonicity in wealth.

Stages 4–5: K_evolve and Z_shock

Stage 4–5

```
K_evolve = WealthChangeStage(layout;  
    wealth_axis=:K,  
    wealth_post=(cell; env) -> begin  
        a0 = env.a0[cell.Z_idx]  
        a1 = env.a1[cell.Z_idx]  
        exp(a0 + a1 * log(cell.K))  
    end,  
)
```

```
Z_shock = MarkovStage(layout; axis=:Z_idx, transition=p.P_Z)
```

K_evolve is a *deterministic* mass-redistribution stage on :K, reading the ALM coefficients from env. WealthChangeStage isn't only for wealth — it redistributes mass along any continuous axis; here we use it on :K.

Compose; declare moments

```
chain = z_shock ◦ income ◦ savings ◦ K_evolve ◦ Z_shock

hh = define_moments!(chain;
  K_supplied=at_end(integrand=:wealth, reduce=sum))
```

- `K_supplied` sums b over the end-of-period Λ — the integral that appears in the K-S iteration.
- Used for the deterministic Aiyagari SS at $Z = 1$ *and* for diagnostics during the K-S loop.

Inner solve: VFI at a candidate ALM

```
a0_init = [log(K_bar), log(K_bar)]
a1_init = [0.0,          0.0]
env0     = make_env(hh; a0=a0_init, a1=a1_init)

ss = solve_steady_state_given_env!(hh, env0;
                                   lambda_tol=1e-5,
                                   lambda_maxiter=50_000)
```

First guess. A *constant* ALM: $K_{t+1} = \bar{K}$ for both Z states ($a_1 \equiv 0$, $a_0 \equiv \log \bar{K}$).

Cost. 400 inner VFI iterations on the 1600-cell grid.

Simulation: why a separate chain

When we simulate we want *realised* (Z_t, K_t) , not the agent's belief.

The chain's K_evolve and Z_shock encode the *belief*: they would push K along the ALM and Z along P_Z , regardless of what actually realised.

Solution. Build a second chain — just the first three stages ($z_shock \circ income \circ savings$) — and drive its (Z, K) from outside.

One simulation step

Maintain Λ_t as a $2D$ distribution on (b, z) . Each period:

1. Compute $K_t = \sum_{b,z} b \Lambda_t[b, z]$.
2. Lift Λ_t to a 4D distribution concentrated at (Z_t, K_t) — share-redistributed between the two adjacent K-grid points.
3. Push the 4D distribution through the sim chain (3 stages).
4. Marginalise out (Z, K) to get Λ_{t+1} on (b, z) .
5. Draw $Z_{t+1} \mid Z_t$ from P_Z .

Seating the simulation chain

```
hh_sim = ks_household_sim(p)
# Run backward on the sim chain with the converged V_ss as
# terminal value, so its savings policy is materialised.
backward!(hh_sim, copy(ss.V), (;))
```

- The sim chain has no K_evolve/Z_shock , so it does not need ALM coefficients in env.
- `backward!` fills the savings policy from the inner-solve V^* .
- Then `ks_simulate` can push Λ forward.

Refit the ALM by OLS

```
function regress_ALM(K_path, Z_idx_path, z_idx, window)
    rows = [t for t in window if Z_idx_path[t] == z_idx]
    x = log.(K_path[rows])
    y = log.(K_path[rows .+ 1])
    # OLS:  $y = a_0 + a_1 * x$  ;  $R^2 = 1 - SS_{res}/SS_{tot}$ 
    ...
    return (; a0, a1, R^2, n)
end

fit_bad = regress_ALM(sim.K_path, sim.Z_idx_path, 1, window_fit)
fit_good = regress_ALM(sim.K_path, sim.Z_idx_path, 2, window_fit)
```

Two regressions (one per Z). Burn-in 500, fit-window the rest.

The outer loop

```
function ks_iterate(p, K_bar,  $\Lambda_0$ ; maxiter=15)
    hh, hh_sim = ks_household(p), ks_household_sim(p)
    a0, a1 = [log(K_bar), log(K_bar)], [0.0, 0.0]
    for it in 1:maxiter
        _ss = solve_steady_state_given_env!(hh, make_env(hh; a0, a1); ...)
        backward!(hh_sim, copy(_ss.V), (;))
        sim = ks_simulate(hh_sim, p, T_sim;  $\Lambda_{\text{init}}=\Lambda_0$ , Z_init_idx=1)
        fit_b = regress_ALM(sim.K_path, sim.Z_idx_path, 1, window_fit)
        fit_g = regress_ALM(sim.K_path, sim.Z_idx_path, 2, window_fit)
        a0_n, a1_n = [fit_b.a0, fit_g.a0], [fit_b.a1, fit_g.a1] $\Delta$ 
        = maximum(abs.(vcat(a0_n .- a0, a1_n .- a1))) $\Delta$ 
        < 1e-3 && return (; a0, a1, sim, iters=it)
        a0 = 0.5 .* a0 .+ 0.5 .* a0_n
        a1 = 0.5 .* a1 .+ 0.5 .* a1_n
    end
end
```

Running it

Convergence behaviour.

- Converges in **11 outer iterations** to $\Delta < 10^{-3}$.
- Inner VFI: ~ 400 iters each outer round, falling as we approach the fixed point.
- Mean K over the post-burn-in window: ≈ 13.70 .

An update speed of 0.5 on the coefficient update prevents the first few rounds from overshooting.

The Accuracy Debate

The converged ALM

After 11 outer iterations:

$$\log K_{t+1} = 0.18 + 0.93 \log K_t \quad (Z = \text{bad}),$$

$$\log K_{t+1} = 0.15 + 0.94 \log K_t \quad (Z = \text{good}).$$

Coefficients on $\log K_t$ are close (0.93 vs 0.94); intercepts differ (0.18 vs 0.15) and pick up the level shift between recession and boom.

Just four numbers describe the dynamics of K across all states of the wealth distribution.

The shocking result: $R^2 \approx 1$

The converged ALM regression:

- $Z = \text{bad}$: $R^2 = 0.99696$.
- $Z = \text{good}$: $R^2 = 0.99862$.

Reading this. A linear function of $\log K_t$ explains the variation in $\log K_{t+1}$ within each Z state *essentially completely*.

Implication. The full distribution Λ_t adds no usable information beyond its mean K_t . The household that knows only K_t forecasts as well as the household that knew all of Λ_t .

The Den Haan critique

R^2 averages residuals over many periods — it can hide *systematic drift*: a small one-step bias compounds over the long run.

Den Haan (2010). A sharper diagnostic. Starting from a common K_0 :

- Compute the true K_t path by aggregating household decisions every period.
- Compute an ALM-only path: $K_{t+1} = \exp(a_0(Z_t) + a_1(Z_t) \log K_t)$, no aggregation.

Compare over hundreds of periods.

What we see

In this calibration, over 800 post-burn-in periods:

$$\max_t \left| \frac{K_t^{\text{ALM}} - K_t^{\text{true}}}{K_t^{\text{true}}} \right| \approx 0.81\%.$$

The ALM-only path tracks the true path to within about 1% throughout the simulation.

Interpretation. The bounded-rationality assumption is essentially exact here. K-S agents forecast as well as fully rational ones — not just on average, but trajectory-by-trajectory.

When K-S works, when it breaks

K-S works when:

- Savings near-linear in wealth.
- Agents relatively patient.
- No large MPC heterogeneity.

K-S can break when:

- Occupation choice / extensive-margin entry.
- Fat-tailed income shocks.
- Non-CRRA $u(c)$ (habits, loss aversion).
- Bankruptcy / default.

What the linear ALM cannot see

The ALM compresses Λ_t into a scalar. Structural changes that move wealth between agents with different MPCs — or shift mass between regimes (constrained vs unconstrained) — without moving K_t much are *invisible* to the ALM.

In the baseline calibration these movements happen to have small impact on K_{t+1} . In richer models they do not.

This is the wedge L8 will exploit.